

```
xulschoolhello/content/browserOverlay.xul
```

- 
- `</code>`

Three lines of code in the whole file – that’s about as easy as it gets. However, this is more declarative than executing code, so it probably doesn’t count. Moving on, we examine a non-critical, but interesting part of the code – the locale. As discussed earlier, this is the language dump for localisation into regional languages for better outreach of the extension. Here, as an example, the structure that’s being used is `root/locale/en-US/browserOverlay.properties`.

- `<code>`
- 
- `xulschoolhello.greeting.label = Hi! How are you?`
- 
- `</code>`

A corresponding controller for that record is the adjacent DTD (Document Type Definition file), which looks something like this:

- `<code>`
- 
- `<!ENTITY xulschoolhello.hello.label "Hello World!">`
- `<!ENTITY xulschoolhello.helloMenu.accesskey "l">`
- `<!ENTITY xulschoolhello.helloItem.accesskey "H">`
- 
- `</code>`

If you look closely, these lines make ample sense for reading by a human (which, incidentally, is one of the great triumphs of XML and all of its descendents). It’s being defined that:

- ▶ `hello.label` property of the object `xulschoolhello` is being set to “Hello World!”
- ▶ `helloMenu.accesskey` property of `xulschoolhello` is being set to “l”
- ▶ `helloItem.accesskey` property of `xulschoolhello` is being set to “H”

It’s important to note that the DTD is an XML spinoff that was created as a set of markup declarations that define a document type for an SGML-